Programación en Java para Algoritmos y Estructuras de Datos

Parte II

Rodrigo Pérez Rodríguez

Universidad Rey Juan Carlos

Contenido

1. Introducción
2. Tipos abstractos de datos
3. Implementación de tipos abstractos de datos
4. Diseño de tipos abstractos de datos
5. Discusión

Introducción

Tipos primitivos vs. referenciados

- Con tipos primitivos (int, double, boolean, char) se escriben programas con números y caracteres.
- Los tipos referenciados permiten acceder a otros tipos de datos: arrays, strings, objetos.
- Las bibliotecas proporcionan tipos como String, arrays, imágenes, sonidos.
- Cada programador/a puede implementar nuevas abstracciones usando orientación a objetos.

Tipos abstractos de datos

Tipos abstractos de datos (TAD)

- Un TAD es un conjunto de valores y operaciones sobre esos valores.
- No requiere orientación a objetos; se puede hacer en lenguajes como C.
- En lenguajes OO como Java, se usan clases para implementar TAD.
- Java es orientado a objetos, por lo que se usa para modelar abstracciones.

Términos clave (1)

Tipo de datos: Conjunto de valores y operaciones que operan sobre dichos valores.

Tipo abstracto de datos (TAD): Tipo de datos cuya representación interna está oculta al código cliente. En Java se usan clases para especificar TAD.

Objeto: Entidad que aloja un valor de un TAD en tiempo de ejecución. Se caracteriza por:

- **Estado:** valor del objeto (variables de instancia)
- **Identidad:** lugar en memoria que distingue un objeto de otro
- **Comportamiento:** operaciones (métodos de instancia)

Términos clave (2)

Clase: Código que especifica en tiempo de compilación la estructura del estado y comportamiento de los objetos que se crearán en tiempo de ejecución.

API: Conjunto de constructores y métodos de instancia públicos, con breve comentario, que especifica el comportamiento de un TAD.

Código cliente: Código fuera de la clase que usa el TAD llamando a constructores y métodos de instancia. No accede directamente a variables de instancia.

Implementación del TAD: Código dentro de la clase: variables de instancia, constructor y métodos.

Diferencia entre clase y TAD

- **Clase:** Es el código Java que escribimos (fichero .java)
  - Especifica la estructura del estado y el comportamiento
  - Puede implementar un TAD o ser una biblioteca de métodos estáticos
- **TAD:** Es un concepto abstracto
  - Conjunto de valores y operaciones sobre esos valores
  - Su representación interna está oculta al cliente
  - Se puede implementar en cualquier lenguaje (no solo Java)
- **Relación:** En Java usamos clases para implementar TAD

Ejemplo: API de Counter

public class	Counter	
	`Counter(String id)`	constructor
`void`	`increment()`	incrementa contador
`int`	`tally()`	devuelve valor
`String`	`toString()`	representación string

Creación de objetos

- Para crear un objeto, se invoca el constructor con `new`.
- Ejemplo: `Counter heads = new Counter("heads");`
- Se reserva memoria, inicializa el objeto y devuelve una referencia.
- El cliente no necesita conocer la implementación, solo la API.

Ejemplo: Creación de objetos

```
public class CounterCreationExample {
    public static void main(String[] args) {
        Counter heads = new Counter("heads");
        Counter tails = new Counter("tails");
        System.out.println(heads);
        System.out.println(tails);
    }
}
```

Invocación de métodos de instancia

- Métodos de instancia operan sobre valores de un TAD.
- Asociados a un objeto concreto.
- Llamada: `objeto.método(parámetros)`
- Ejemplo: `heads.increment(); heads.tally();`

Ejemplo: Uso de Counter

```java
public class CounterUsageExample {
    public static void main(String[] args) {
        Counter heads = new Counter("heads");
        Counter tails = new Counter("tails");

        heads.increment();
        tails.increment();

        int h = heads.tally(); // h = 1
        int t = tails.tally(); // t = 1

        System.out.println("heads: " + h);
        System.out.println("tails: " + t);
    }
}
```

Interfaz de Programación de Aplicaciones (API)

- El **API** (Application Programming Interface) es el conjunto de métodos **públicos** que los programadores pueden usar para interactuar con un tipo abstracto de datos (TAD).
- Define **qué** se puede hacer con el TAD, pero **no cómo** se implementa internamente.
- Los clientes del TAD usan el API sin conocer los detalles de implementación.
- Ejemplo: Para usar un contador, solo necesitamos saber cómo crear uno y cómo incrementarlo, no cómo almacena el valor internamente.

Código cliente: Flips.java

```
import edu.princeton.cs.algs4.StdRandom;

public class Flips {
    public static void main(String[] args) {
        int T = Integer.parseInt(args[0]);
        Counter heads = new Counter("heads");
        Counter tails = new Counter("tails");
        for (int t = 0; t < T; t++)
            if (StdRandom.bernoulli(0.5))
                heads.increment();
            else tails.increment();
        System.out.println(heads);
        System.out.println(tails);
    }
}
```

Aliasing

- Asignación copia la referencia, no el objeto (aliasing).
- Ejemplo: `Counter c2 = c1;` hace que c1 y c2 referencien el mismo objeto.
- Modificar uno afecta al otro.
- Fuente común de errores.

Ejemplo de aliasing

```java
public class AliasingExample {
    public static void main(String[] args) {
        Counter c1 = new Counter("ones");
        Counter c2 = c1;  // aliasing: c2 referencia el mismo objeto que c1
        c1.increment();
        c2.increment();
        System.out.println(c1);  // muestra 2 ones
        System.out.println(c2);  // muestra 2 ones (mismo objeto)
    }
}
```

Objetos como parámetros

- Se pasan referencias por valor (!).
- El método puede modificar el objeto, pero no cambiar la referencia del parámetro.

Objetos devueltos

- Se devuelve una referencia al objeto.
- Permite devolver múltiples valores encapsulados en el objeto.

Ejemplo: Objeto devuelto

```
import edu.princeton.cs.algs4.StdRandom;

public class FlipsMax {
    public static Counter max(Counter x, Counter y) {
        if (x.tally() > y.tally()) return x;
        else return y;
    }
    public static void main(String[] args) {
        int T = Integer.parseInt(args[0]);
        Counter heads = new Counter("heads");
        Counter tails = new Counter("tails");
        for (int t = 0; t < T; t++)
            if (StdRandom.bernoulli(0.5))
                heads.increment();
            else tails.increment();
        Counter winner = max(heads, tails);
        System.out.println(winner + " wins");
    }
}
```

Ejemplo: Referencia perdida

```java
import edu.princeton.cs.algs4.Counter;
import edu.princeton.cs.algs4.StdRandom;

public class FlipsBreakReference {
    private static Counter breakReference(Counter c) {
        c.increment();
        c = new Counter("new counter");  // rompe la referencia original
        return c;
    }

    private static Counter doNotBreakReference(Counter c) {
        c.increment();
        return c;
    }

    public static void main(String[] args) {
        Counter counter = new Counter("counter");

        System.out.println("Before breakReference: " + counter);
        counter = breakReference(counter);
        System.out.println("After breakReference: " + counter);

        System.out.println("Before doNotBreakReference: " + counter);
        counter = doNotBreakReference(counter);
        System.out.println("After doNotBreakReference: " + counter);
    }
}
```

Arrays y Strings son objetos

- Todos los tipos no primitivos son objetos.
- Arrays y Strings tienen sintaxis especial.
- **Se almacenan referencias:** Cuando asignamos un array o String a una variable, almacenamos una **referencia** al objeto en memoria, no el objeto completo.
- Esto permite **eficiencia** (no se copian objetos grandes) y **aliasing** (múltiples variables pueden referenciar el mismo objeto).
- Ejemplo: `int[] a = {1, 2, 3}; int[] b = a;` hace que `a` y `b` referencien el mismo array.

Ejemplo: Array de objetos (ver con debugger)

```
import edu.princeton.cs.algs4.StdRandom;

public class Rolls {
    public static void main(String[] args) {
        int T = Integer.parseInt(args[0]);
        int SIDES = 6;
        Counter[] rolls = new Counter[SIDES+1];
        for (int i = 1; i <= SIDES; i++)
            rolls[i] = new Counter(i + "s");
        for (int t = 0; t < T; t++) {
            int result = StdRandom.uniform(1, SIDES+1);
            rolls[result].increment();
        }
        for (int i = 1; i <= SIDES; i++)
            System.out.println(rolls[i]);
    }
}
```

Programación de TAD con POO

- Implementar TAD en clases Java.
- Crear objetos con `new`.
- Manipular con métodos de instancia.
- Objetos tienen estado, comportamiento e identidad.

TAD estándar en Java

- Miles de TAD ya definidos.
- `java.lang.*` sin import.
- Otros requieren import (java.awt, java.net, etc.).
- En la asignatura: In, Out, Draw, etc.

Ejemplo: API de Point2D

public class	Point2D	
	Point2D(double x, double y)	constructor
double	x()	coordenada x
double	y()	coordenada y
double	r()	radio polar
double	theta()	ángulo polar
double	distanceTo(Point2D that)	distancia
void	draw()	dibuja punto

Ejemplo: API de Interval1D

public class	Interval1D	
	`Interval1D(double min, double max)`	constructor
double	`min()`	extremo izquierdo
double	`max()`	extremo derecho
double	`length()`	longitud

Ejemplo: API de Date

public class	Date	
	`Date(int month, int day, int year)`	constructor
	`Date(String date)`	constructor desde string
`int`	`month()`	mes
`int`	`day()`	día
`int`	`year()`	año
`String`	`toString()`	representación string
`boolean`	`equals(Object that)`	¿misma fecha?

Ejemplo: Implementación de Date

```
public class Date {
    private final int month;
    private final int day;
    private final int year;

    public Date(int m, int d, int y) {
        month = m; day = d; year = y;
    }

    public int month() { return month; }
    public int day()   { return day; }
    public int year()  { return year; }

    public String toString() {
        return month() + "/" + day() + "/" + year();
    }

    public boolean equals(Object x) {
        if (this == x) return true;
        if (x == null) return false;
        if (this.getClass() != x.getClass()) return false;
        Date that = (Date) x;
        if (this.day   != that.day)   return false;
        if (this.month != that.month) return false;
        if (this.year  != that.year)  return false;
        return true;
    }
}
```

Ejemplo: API de Transaction

public class	Transaction	
	`Transaction(String who, Date when, double amount)`	constructor
	`Transaction(String transaction)`	constructor de
`String`	`who()`	nombre del cli
`Date`	`when()`	fecha
`double`	`amount()`	cantidad
`String`	`toString()`	representación
`boolean`	`equals(Object that)`	¿misma transa

Ejemplo: API de String

public class	String	
	`String()`	constructor (string vacío)
`int`	`length()`	longitud
`char`	`charAt(int i)`	carácter en posición i
`int`	`indexOf(String pat)`	1ª ocurrencia de pat
`String`	`concat(String that)`	concatena that al final
`String`	`substring(int i, int j)`	substring (i a j-1)
`String[]`	`split(String delim)`	separa por delim
`boolean`	`equals(String that)`	¿mismo string?

Ejemplos de operaciones con String

```
String a = "now is";
String b = "the time ";
String c = "to";

a.length()            // 6
a.charAt(4)           // 'i'
a.concat(c)           // "now is to"
a.indexOf("is")       // 4
a.substring(2, 5)     // "w i"
a.split(" ")[0]       // "now"
a.split(" ")[1]       // "is"
b.equals(c)           // false
```

Ejemplo: ¿es palíndromo?

```
public static boolean isPalindrome(String s) {
    int n = s.length();
    for (int i = 0; i < n/2; i++) {
        if (s.charAt(i) != s.charAt(n-1-i)) {
            return false;
        }
    }
    return true;
}
```

Ejemplo: Concatenación de Strings

```java
public class Concat {

    public static void main(String[] args) {
        String s = "Hello";
        s = s + " World";
        s = s.concat("!");
        System.out.println(s);
    }
}
```

Implementación de tipos abstractos de datos

Implementación del TAD Counter

```
public class Counter {
    private final String name;
    private int count = 0;

    public Counter(String id) { name = id; }
    public void increment() { count++; }
    public int tally() { return count; }
    public String toString() { return count + " " + name; }

    public static void main(String[] args) {
        Counter heads = new Counter("heads");
        Counter tails = new Counter("tails");
        heads.increment(); heads.increment(); tails.increment();
        System.out.println(heads + " " + tails);
        System.out.println(heads.tally() - tails.tally());
    }
}
```

Variables de instancia

- Definen los valores del tipo de datos (estado del objeto)
- Ámbito: desde definición hasta final de la clase
- Modificador `final`: valor no modificable tras inicialización

Modificadores de acceso

`private`: Solo accesible dentro de la misma clase

- Oculta representación interna al código cliente
- Recomendado para variables de instancia (encapsulación)

`public`: Accesible desde cualquier clase

- Usado para métodos del API público
- No recomendado para variables de instancia

`protected`: Accesible en la misma clase, subclases y mismo paquete

- Usado para permitir acceso a subclases
- Rompe parcialmente la encapsulación

Constructores

- Establece identidad del objeto (en memoria)
- Inicializa variables de instancia
- Mismo nombre que la clase
- No se especifica tipo de retorno
- Se pueden sobrecargar (múltiples constructores con diferentes parámetros)
- Java proporciona constructor por defecto si no se escribe ninguno

Métodos de instancia

- Especifican las operaciones del TAD
- Tipo devuelto, firma (nombre y parámetros), cuerpo
- Parámetros se pasan por valor (se copia el valor)
- Modificador `public`: disponible en API para clientes
- Modificador `private`: solo para uso interno del TAD

Ámbito de variables

Parámetros formales: Ámbito es todo el método

Variables locales: Desde declaración hasta final del bloque

Variables de instancia: Ámbito es toda la clase

Ambigüedad: Usar `this.variableDeInstancia` para diferenciar

Ambigüedad con variables de instancia

```java
public class ThisExample {
    private int valor;

    public ThisExample(int valor) {
        this.valor = valor; // this.valor: variable de instancia
    }                       // valor: parámetro

    public ThisExample() { valor = 0; }

    public int suma(int valor) {
        return this.valor + valor; // this.valor: variable instancia
    }                              // valor: parámetro

    public int resta(int v) {
        return valor - v; // sin this porque no hay ambigüedad
    }
}
```

Diseño de tipos abstractos de datos

Encapsulación

- Ocultar representación de valores e implementación de operaciones
- Facilita desarrollo independiente de TAD y código cliente
- Permite cambiar implementación sin afectar clientes
- Herramienta fundamental para programación modular

Diseño de APIs: errores a evitar

- Demasiado difícil de implementar
- Demasiado difícil de usar
- Demasiado limitada u omite métodos necesarios
- Demasiado extensa: incluye métodos innecesarios
- Demasiado general o demasiado específica
- Dependiente de representación particular

Objetivo: API lo más sencilla posible, proporcionar solo lo necesario

Ejemplo: Whitelist (independiente de implementación de StaticSETofInts)

```java
import edu.princeton.cs.algs4.*;

public class Whitelist {
    public static void main(String[] args) {
        In in = new In(args[0]);
        int[] a = in.readAllInts();
        StaticSETofInts set = new StaticSETofInts(a);
        while (!StdIn.isEmpty()) {
            // Leer key
            int key = StdIn.readInt();
            // Mostrar key si no está en la lista blanca
            if (!set.contains(key)) {
                StdOut.println(key);
            }
        }
    }
}
```

Herencia de interfaces

- Varias clases implementan mismo conjunto de firmas (interfaz)
- Palabra reservada: `implements`
- No implica compartir código o variables de instancia
- Relación entre especificaciones de clases
- Ejemplos: `Comparable`, `Comparator`, `Iterable`

Ejemplo: Definición de interfaz

```
public interface Datable {
    int month();
    int day();
    int year();
}
```

Ejemplo: Implementación de interfaz

```
public class Date implements Datable {
    private final int month;
    private final int day;
    private final int year;

    public Date(int m, int d, int y) {
        month = m; day = d; year = y;
    }

    public int month() { return month; }
    public int day()   { return day; }
    public int year()  { return year; }

    // otros métodos...
}
```

Herencia de implementaciones

- Reutiliza código de clase padre (superclase)
- Palabra reservada: `extends`
- Subclase hereda métodos y variables de instancia
- Variables/métodos `public` o `protected` son heredados
- **Toda clase en Java hereda de** `Object`
- Métodos importantes: `toString()`, `equals()`, `hashCode()`

Ejemplo: Clase base Counter

```
public class Counter {
    private int count = 0;

    public void increment() { count++; }
    public int tally() { return count; }
    public String toString() { return "" + count; }
}
```

Ejemplo: Herencia con extends

```java
import edu.princeton.cs.algs4.StdOut;

public class CounterWithName extends Counter {
    private final String name;

    public CounterWithName(String id) { name = id; }

    public String toString() {
        return super.toString() + " " + name; // llama al método heredado
    }

    public static void main(String[] args) {
        CounterWithName heads = new CounterWithName("heads");
        heads.increment();  // método heredado de Counter
        StdOut.println(heads);  // llama a toString()
    }
}
```

Métodos heredados de Object

En Java, **toda clase hereda de Object**. Por tanto, toda clase hereda estos métodos:

	método	**descripción**
`Class`	`getClass()`	¿de qué clase es `this`?
`String`	`toString()`	representación como string de `this`
`boolean`	`equals(Object that)`	¿es `this` igual a `that`?
`int`	`hashCode()`	código hash de `this`

Importante: En la mayoría de las clases se deben reemplazar las implementaciones de `toString()`, `equals()` y `hashCode()`.

Tipos envoltorio (wrapper)

Tipo primitivo	Tipo envoltorio
int	Integer
double	Double
boolean	Boolean
char	Character

Utilidad de tipos envoltorio

- Permiten usar tipos primitivos como objetos
- Necesarios para usar tipos primitivos en colecciones genéricas (`ArrayList<Integer>`)
- Proporcionan métodos útiles: `Integer.parseInt()`, `Double.toString()`, etc.
- Implementan `equals()`, `hashCode()`, `compareTo()`
- Permiten valores `null` (tipos primitivos no pueden ser `null`)

Autoboxing: primitivo → wrapper

Conversión automática de tipo primitivo a objeto envoltorio:

```java
// Autoboxing: int -> Integer
Integer a = 23;  // equivale a: Integer a = Integer.valueOf(23);

// Autoboxing en método println
System.out.println("número de horas: " + 23);
// Se convierte 23 a Integer y se llama a toString()

// Autoboxing en colecciones
ArrayList<Integer> list = new ArrayList<>();
list.add(42);  // autoboxing: int 42 -> Integer
```

Auto-unboxing: wrapper → primitivo

Conversión automática de objeto envoltorio a tipo primitivo:

```
// Auto-unboxing: Integer -> int
Integer a = 23;
int b = a;  // equivale a: int b = a.intValue();

// Auto-unboxing en operaciones aritméticas
Integer x = 10;
Integer y = 20;
int suma = x + y;  // auto-unboxing de x e y, luego suma

// Auto-unboxing en comparaciones
Integer n = 100;
if (n > 50) {  // auto-unboxing: n.intValue() > 50
    System.out.println("mayor que 50");
}
```

Comparación de igualdad entre objetos

- `a == b` compara referencias (direcciones), no valores
- `a.equals(b)` compara valores (estado de objetos)
- Método `equals()` debe cumplir:
  - Reflexiva: `x.equals(x)` es true
  - Simétrica: `x.equals(y)` $\iff$ `y.equals(x)`
  - Transitiva
  - Consistencia
  - `x.equals(null)` debe devolver false

Ejemplo: Implementación de equals() en Date

```
public class Date {
    private final int month;
    private final int day;
    private final int year;

    // ... constructores y otros métodos ...

    public boolean equals(Object x) {
        if (this == x) return true;               // ¿misma referencia?
        if (x == null) return false;              // ¿es null?
        if (this.getClass() != x.getClass())      // ¿misma clase?
            return false;
        Date that = (Date) x;                     // cast a Date
        if (this.day   != that.day)   return false;
        if (this.month != that.month) return false;
        if (this.year  != that.year)  return false;
        return true;                              // son iguales
    }
}
```

Usa este método como plantilla para implementar equals() en tus clases.

Ejemplo: Uso de equals() vs ==

```
public class EqualsExample {

    public static void main (String args[]) {

        Date d1 = new Date(1, 2, 2020);
        Date d2 = new Date(1, 2, 2020);

        System.out.println(d1.equals(d2));  // true (mismo estado)
        System.out.println(d1 == d2);       // false (referencias distintas)

    }

}
```

Importante: `equals()` compara valores, `==` compara referencias.

Gestión de memoria e inmutabilidad

- **Recolección de basura:** Java reclama automáticamente memoria de objetos huérfanos
- **Tipo inmutable:** valor del objeto nunca cambia tras construcción
- Modificador `final` de Java: asegura inmutabilidad de variables de instancia
- **Ventajas inmutabilidad:** código más fácil de depurar, aliasing no es problema
- **Coste:** se crean más objetos; una vez se ha creado uno, no se puede modificar (mitigado por recolector de basura)

Inmutabilidad de Date

```
public class Date {
    private final int month;  // final: no se puede modificar
    private final int day;    // final: no se puede modificar
    private final int year;   // final: no se puede modificar

    public Date(int m, int d, int y) {
        month = m;
        day = d;
        year = y;
    }

    // NO hay métodos setMonth(), setDay(), setYear()
    // Una vez creado, el objeto Date es inmutable
}
```

Consecuencia: Para cambiar una fecha, hay que crear un nuevo objeto `Date`.

Ejemplo: Vector inmutable (correcto)

```
public class Vinmutable {
    private int[] coords;

    public Vinmutable(int [] a) {
        coords = new int[a.length];  // Copia defensiva
        for (int i = 0; i < a.length; i++) {
            coords[i] = a[i];
        }
    }

    public String toString() {
        String s = "";
        for (int i = 0; i < coords.length; i++) {
            s = s + coords[i] + " ";
        }
        return s;
    }
}
```

Clave: El constructor copia el array. Cambios externos no afectan al objeto.

Ejemplo: Vector mutable (incorrecto)

```java
public class Vmutable {
    private final int[] coords;  // final NO garantiza inmutabilidad

    public Vmutable(int [] a) {
        coords = a;  // ¡Comparte referencia!
    }

    public String toString() {
        String s = "";
        for (int i = 0; i < coords.length; i++) {
            s = s + coords[i] + " ";
        }
        return s;
    }
}
```

Problema: Aunque `coords` es `final`, el contenido del array puede cambiar externamente.

Excepciones

- `ArithmeticException`: división por cero
- `ArrayIndexOutOfBoundsException`: índice inválido
- `NullPointerException`: referencia null
- `OutOfMemoryError`: sin memoria
- `StackOverflowError`: demasiadas llamadas recursivas
- Se pueden capturar con `try { ... } catch(...) { ... }`

Ejemplo: Excepción no capturada

```java
import edu.princeton.cs.algs4.StdOut;

public class Excepciones1 {
    public static void main (String[] args) {
        int i = 1;
        int j = 0;

        i = i/j;  // ArithmeticException: / by zero

        StdOut.println("Después de excepción");  // No se ejecuta
    }
}
```

Resultado: El programa termina abruptamente con `ArithmeticException`.

Ejemplo: Excepción capturada

```
import edu.princeton.cs.algs4.StdOut;

public class Excepciones2 {
    public static void main (String[] args) {
        int i = 1;
        int j = 0;

        try {
            i = i/j;
        }
        catch (java.lang.ArithmeticException e) {
            System.err.println("División entre 0");
            // System.exit(1);
        }

        StdOut.println("Después de excepción");  // SÍ se ejecuta
    }
}
```

Resultado: Se captura la excepción, se imprime el error y el programa continúa.

Aserciones

```
assert index >= 0;
```

- Verifican suposiciones del código
- Si expresión es false, programa termina con mensaje de error
- Habilitar con: `java -ea MiPrograma`
- Útiles para depuración
- No ejecutar en producción (omitir opción `-ea`)

Ejemplo: Aserciones

```java
public class Aserciones {
    public static void main (String[] args) {
        int i = 1;
        int j = 2;

        assert i > j;  // Falla: 1 no es mayor que 2

        System.out.println("Después de aserción");
    }
}
```

Sin -ea: La aserción se ignora, el programa continúa.
Con -ea: Lanza `AssertionError` y termina el programa.

Discusión

Preguntas para discusión

- ¿Qué diferencia hay entre tipos primitivos y referenciados?
- ¿Por qué es importante ocultar la implementación en un TAD?
- ¿Qué es el aliasing y por qué puede causar errores?
- ¿Cómo se pasan objetos como parámetros en Java?
- ¿Qué ventajas tiene la programación orientada a objetos para TAD?

¿Qué diferencia hay entre tipos primitivos y referenciados?

- **Primitivos (int, double, boolean, char):** Almacenan valores directamente
- **Referenciados (objetos, arrays, String):** Almacenan referencias a objetos en memoria
- **Consecuencia:** Asignación de objetos crea aliasing, no copias
- **Ejemplo:** `int x = 5;` vs. `Counter c = new Counter("test");`

¿Por qué es importante ocultar la implementación en un TAD?

- **Abstracción:** Clientes usan el TAD sin conocer detalles internos
- **Mantenibilidad:** Cambios de implementación no afectan código cliente
- **Modularidad:** Separación clara entre interfaz (API) e implementación
- **Robustez:** Implementación interna puede cambiar sin romper contratos

¿Qué es el aliasing y por qué puede causar errores?

- **Aliasing:** Dos variables referencian el mismo objeto en memoria
- **Causa:** Asignación de referencias (`Counter c2 = c1;`)
- **Problema:** Modificar uno afecta al otro inesperadamente
- **Ejemplo:** Cambios en c2 afectan c1, causando bugs difíciles de detectar

¿Cómo se pasan objetos como parámetros en Java?

- **Por referencia:** Se pasa la referencia al objeto, no una copia
- **Consecuencia:** El método puede modificar el objeto original
- **Limitación:** No se puede cambiar la referencia del parámetro
- **Ejemplo:** `void increment(Counter c) { c.increment(); }` modifica el contador original

¿Qué ventajas tiene la programación orientada a objetos para TAD?

- **Encapsulación:** Estado y comportamiento juntos en una clase
- **Herencia:** Reutilización de código mediante jerarquías de clases
- **Abstracción:** Ocultar complejidad detrás de interfaces simples